SILENT CONTEXT

Expert advice lives in chats and emails, and stays there.

The next person solves the same fault **from zero.**

Elisa's challenge: capture what field engineers and network experts learn, without giving those people extra work; tie it to the right network entities with its source and period of validity; and let an agent use it the next time a similar case appears.

Petri Korhonen · github.com/pkorrrhonen-cmd/silent-context · MIT

Veikko Vikkelä Field Engineer · 09:14
Changed the SFP on transmission node TRN-042. Link is up and alarms cleared, but traffic is still not coming back.

Erkki Expert Network Expert · 09:19
Check whether the traffic counters are actually increasing. If they stay at zero even though the interface is UP, it's probably the issue I've seen before.

Veikko Vikkelä Field Engineer · 09:21
Yes. Link UP. RX power looks normal. Counters basically zero.

Erkki Expert Network Expert · 09:22
That's it. Disable the interface, wait a little, then enable it again. Don't reboot the whole unit unless that fails.

Veikko Vikkelä Field Engineer · 09:26
Is this documented somewhere?

Erkki Expert Network Expert · 09:27
Not that I know of. We've just learned it over the years.

Teams excerpts · Elisa's example file, unchanged

THE ANSWER

Silent context turns those messages into claims an agent can use

READ AS WRITTEN

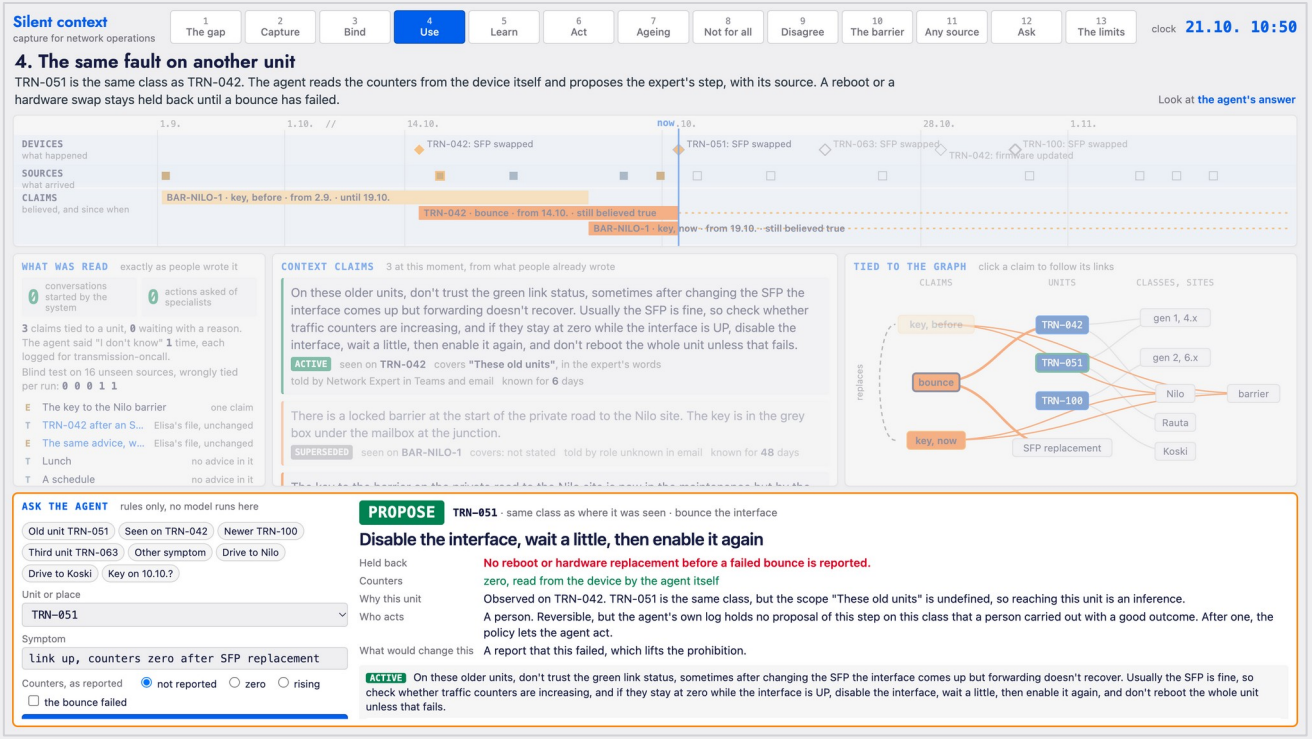
Teams threads and emails, exactly as they are. Nobody fills in anything for the system.

TIED TO THE NETWORK

A context claim on the unit, with its source, the expert's own words on which units it covers, and two clocks: when it holds in the world, and when we learned it.

ONE CALL FOR THE AGENT

`claims_for` answers with a next step and its sources: **propose**, **check first**, or **I don't know** with who to ask.



The page: thirteen scenes, no model and no key. Every answer on it is computed when asked, and the time axis shows what was known when.

CAPTURE AND BIND

The model reads; code binds, or waits with a reason

READ

One model call per thread reports what the text says. An identifier counts only if it is literally in the text.

BIND

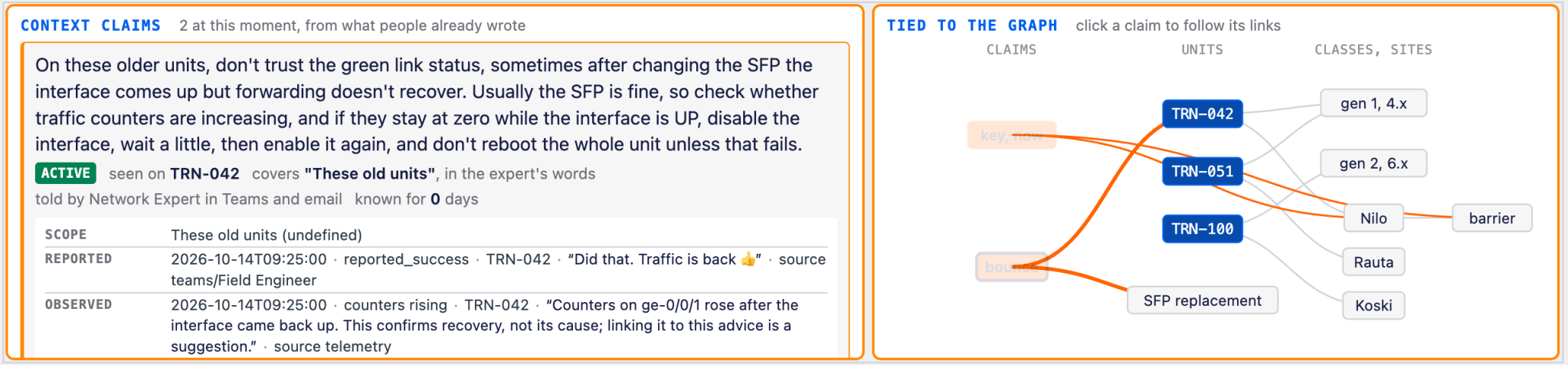
Code ties the claim to the graph. The model never picks an entity.

WAIT

An unclear binding waits with its reason: no unit named, several could fit, an unknown id, a contradicting context. It may wait forever. The system never asks anyone.

MERGE

One person's accounts become one claim with several sources. A relay is a source, not a telling.



THE DATA MODEL

A claim keeps its words, its scope and two clocks

WORDS

Claim text in the speaker's words, with “sometimes” and “usually” kept.

ENTITY AND SCOPE

The entity it was seen on, and apart from it the scope: wording verbatim, status confirmed or undefined.

`seen on TRN-042 · covers “These old units” (undefined)`

TWO CLOCKS

World time and information time apart. Unknown time stays unknown. No age limit is derived from “installed around 2019”.

`from 14.10. · still believed true`

EVIDENCE

Events with a date and a link: told, reported success, reported failure, disputed, superseded. Rendered from a template: no counts, no percentages, no figure per person.

STATUS

Computed at read time for a claim and a unit, never stored.

`ACTIVE · SUPERSEDED · DISPUTED`

THE GRAPH

Entities: units, classes, sites, a barrier, a procedure type and groups. Exported as graph.json and graph.cypher; the Cypher loads into Neo4j 5 with no edits.

THE AGENT

Three tiers from rules, one prohibition in code, no model

PROPOSE

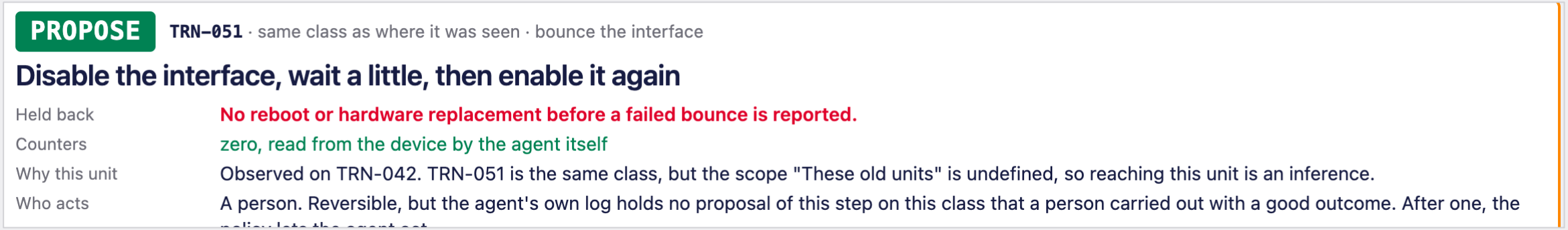
A success is reported on this unit's class, no disagreement, and the claim's own check holds.

CHECK FIRST

Only told, in doubt, a guess, or disputed.

I DON'T KNOW

With the group to ask, and the gap logged.



The page's fourth scene: the same fault on TRN-051, the same class as TRN-042. The agent reads the counters from telemetry itself when a recent sample exists, proposes the expert's step with its source, and holds back a reboot or a hardware replacement until a failed bounce is reported.

What a person did, and the outcome, fill in from the device events. Nobody reports anything. That record is what the next slide's policy reads.

WHO ACTS

A policy says who does the next step. The agent itself, once a success is on record

Step	Reversible	Needs	Who acts
Turn the port off and on	yes	proposed on this class before and carried out with the counters rising; no disagreement; the counters read from the device by the agent	**the agent itself**
Reboot	no	a failed lighter fix reported	**a person**
Replace a part	no	the same	**a person**
Drive to a site	no		**a person**

THE AGENT ACTED **TRN-063** · under the policy, in the synthetic network

The agent proposed	Disable the interface, wait a little, then enable it again
The policy said	The agent itself. Reversible; proposed on TRN-051 on 21.10., carried out by a person, and the counters rose; no disagreement on record; the counters here read from the device by the agent itself.
The agent did	Disabled and enabled ge-0/0/1, seen on the device at 2026-10-25T09:42:00
The outcome	Counters rising afterwards
Still with a person	**A reboot or a hardware replacement: not reversible, so the policy never gives them to the agent.**

The page's sixth scene: the same fault on a third unit of the same class, after the agent's own log shows TRN-051's proposal carried out with the counters rising. The decision is real and computed when asked; the action is an event in the synthetic stream, with no write path to any device.

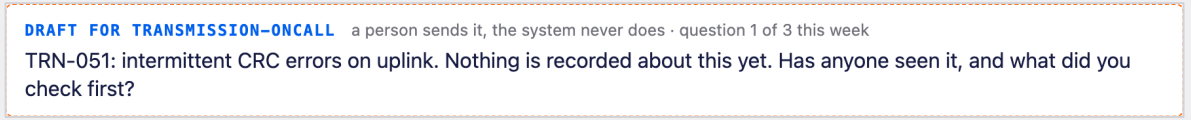
ANY SOURCE, AND THE QUESTION

The same reader for a ticket and a thread in Finnish; a drafted question when it does not know

ANY SOURCE, ANY LANGUAGE

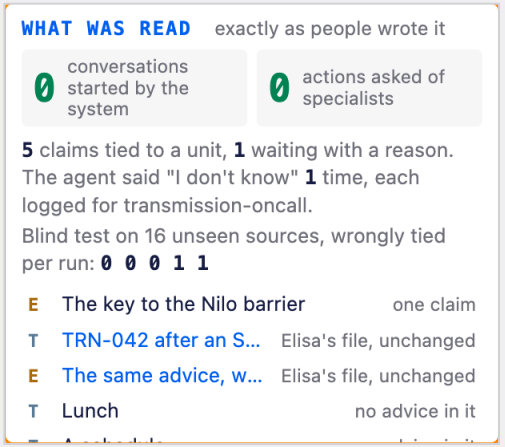
A ticket export's worklog and a Teams thread in Finnish are read by the same reader as Elisa's Teams export and email, and become the same kind of claim, in the speaker's own words and language. The ticket's closing note adds an outcome to advice told in a chat. Ask about the Koski gate and the answer is in Finnish, because that is what was said.





WHEN IT DOES NOT KNOW

It writes the question for the person on call, who sends it. The system itself sends nothing. A budget of three questions a week per group shows when the asking would become the extra work it replaces.



WHAT WAS MEASURED

Everything below runs with no key, from the recorded answers

160

tests pass

The data set's shape, the store's rules, extraction and binding against the truth set, the agent's acceptance cases, the policy on who acts, the page, the MCP wrapper.

13 / 13

truth set sources found

13 sources: merged 9/9, entity 5/5, scope 5/5, time 9/9, evidence 8/8. The second day's two rows were written before their sources were read.

1 of 16

confidently wrong, held-out binding set

16 synthetic sources the rule author has not read. Five repeats with the prompt unchanged: 0, 0, 0, 1, 1, the same file each time.

21 of 21

acceptance rows hold

The build plan's cases A0 to A11 and A12 for the policy, written by app.accept from the recorded answers. A test keeps every computed row passing.

Passed

clean install from the README

git archive into an empty folder, .env with no values, every command in the README runs with no key, on 7.11.2026.

No edits

Cypher export loaded into Neo4j 5

At noon on 7.11.: 5 claims, 14 entities, 14 evidence events and 9 sources as nodes, all ten edge types; one query answered the agent's question from the graph. The data grew that evening; the load was not repeated.

These numbers describe this synthetic data only. Not tested: extraction on real conversations, outcomes read from real chat, and the class binding on anything but this data.

ELISA'S QUESTION

Is the information fact? The answer is a claim's origin, not a score

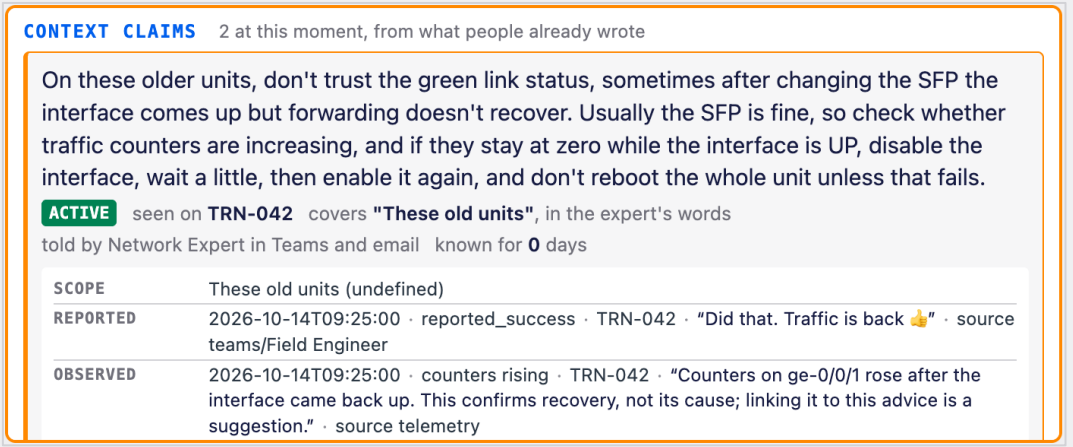
The answer is a claim's origin, what was reported about it, and when it was last confirmed. Not a score for the person.

WHY NO PERSON SCORES

- An employee assessment it was never meant to be.
- Personal data it does not need.
- A source that dries up once it is graded.

WHY NO COUNTS

“Confirmed twice” invites a trust the data does not carry. The events are listed; the reader counts if they want to.



One claim on the page: its words, where it was seen, which units it covers in the expert's own words, who told it and where, how long it has been known, and the evidence as events with their source.

WHAT IS REAL, WHAT IS SIMULATED

The mechanism is real; the network behind it is not

REAL

- Elisa's two files, unchanged; a test holds their checksums.
- The model's reading of every source, recorded once and replayed; not called at demo time.
- Binding, dating and merging, in code, live on every rebuild of the store.
- The agent: claims_for and access_for, rules and a template, no model.
- The exports: graph.json and graph.cypher, written from the store.

SIMULATED

- Every other source: nine threads and emails written for this demo, and on the second day a ticket export and a thread in Finnish.
- The dates: Elisa's files carry none, so a manifest gives each source a received time.
- The event stream and the telemetry; there is no network behind them.
- The graph: four units, two classes, four sites, a barrier, a procedure type, five groups.
- The demo clock and the channels. No real network, channel or person.

NOT BUILT, ON PURPOSE

- Sending the question to the expert: it is drafted, with a budget; a person sends it.
- Matching Elisa's own inventory graph; its schema is not known here.
- A semantic symptom match; symptoms are matched by plain words.
- Acting on a real network: the policy's action is an event in the synthetic stream, with no write path to any device.
- Live connectors to Teams and Slack, a chat with the agent, search by meaning: each a connector or a model away.

WHAT COMES NEXT

Sending the question, more of the policy, live sources

SENDING THE QUESTION

The question is drafted now, with a budget of three a week per group; a person sends it. The next step is a connector that sends it when the budget and the situation allow, a 20-second call when Erkki is driving away from the site. Synthetic data cannot show it is not extra work, so it is a step for real data.

MORE OF THE POLICY

The first table covers one reversible step. Next: more action classes, a limit on blast radius, and a review of the agent's own actions, so that certainty stays necessary and never sufficient.

LIVE SOURCES

Connectors to Teams and Slack that read as the messages arrive, a ticketing system's own export, and a chat with the agent on top of the same calls.

FOR PRODUCTION AT ELISA

Schema matching against the inventory graph, Finnish inflections from the inventory or a lemmatiser, and a semantic symptom match.

THE LIMITS, SAID PLAINLY

What this demo does not claim

01 The system never sends a message itself. It drafts; a person sends.

02 It shows the origin of a claim and what was reported about it, not the truth.

03 It scores claims, not people: no personal assessment and no figure shown per person.

04 Telling apart which units a claim applies to, before someone narrows it down, rests on synthetic data.

05 The network here is simulated, and so is every file but Elisa's two; the agent acts only in that stream.

06 Left out on purpose: live connectors, a chat with the agent, search by meaning, Elisa's inventory graph.

github.com/pkorrrhonen-cmd/silent-context MIT Petri Korhonen